ФГАОУ ВО «Омский государственный технический университет»
Кафедра «Автоматизированные системы обработки информации и управления»

ОТЧЁТ

по дисциплине «СУБД»
на тему: «Работа с СУБД PostgreSQL»
студента Ерилова Николая Алексеевича группы ИВТ-213

Пояснительная записка
Шифр работы ЛР-02068999-43-ИВТ-213-9 ПЗ
Направление 09.03.01 Информатика и вычислительная техника

Старший преподаватель М.П. Маркова

Студент Н.А. Ерилов

Омск 2023

Введение

Цель и краткое описание работы: Разработать прикладное программное обеспечение деятельности депо по ремонту пассажирских вагонов. Депо выполняет несколько видов ремонта. Деповской ремонт – после пробега вагоном 450 тыс. км или два года эксплуатации (что наступит раньше). ТО-2 – подготовка вагона к зимним или летним условиям эксплуатации. ТО-3 – текущее обслуживание – после пробега 150 тыс. км или один год эксплуатации. Текущий ремонт – круглосуточно, при котором ремонтируются вагоны всех дорог России. Основные причины поступления вагона в текущий ремонт: неисправность колесной пары, неисправность буксового узла и т. д. Каждый вагон имеет уникальный номер. Тип вагона также имеет значение при ремонте: купейный, СВ, плацкартный, почтовый, багажный. Каждый вагон приписан к дирекции по обслуживанию пассажиров (ДОП-1, ДОП-2, ДОП-3 и т. д.). За высокое качество ремонта члены бригады получают премию.

Результаты выполнения лабораторных работ

Лабораторная работа № 1

«Проектирование базы данных с использованием ER-технологии»

Описание работы: Для выданного варианта предметной области необходимо:

• выделить сущности, атрибуты и связи предметной области;

• построить модель данных в одной из принятых нотаций.

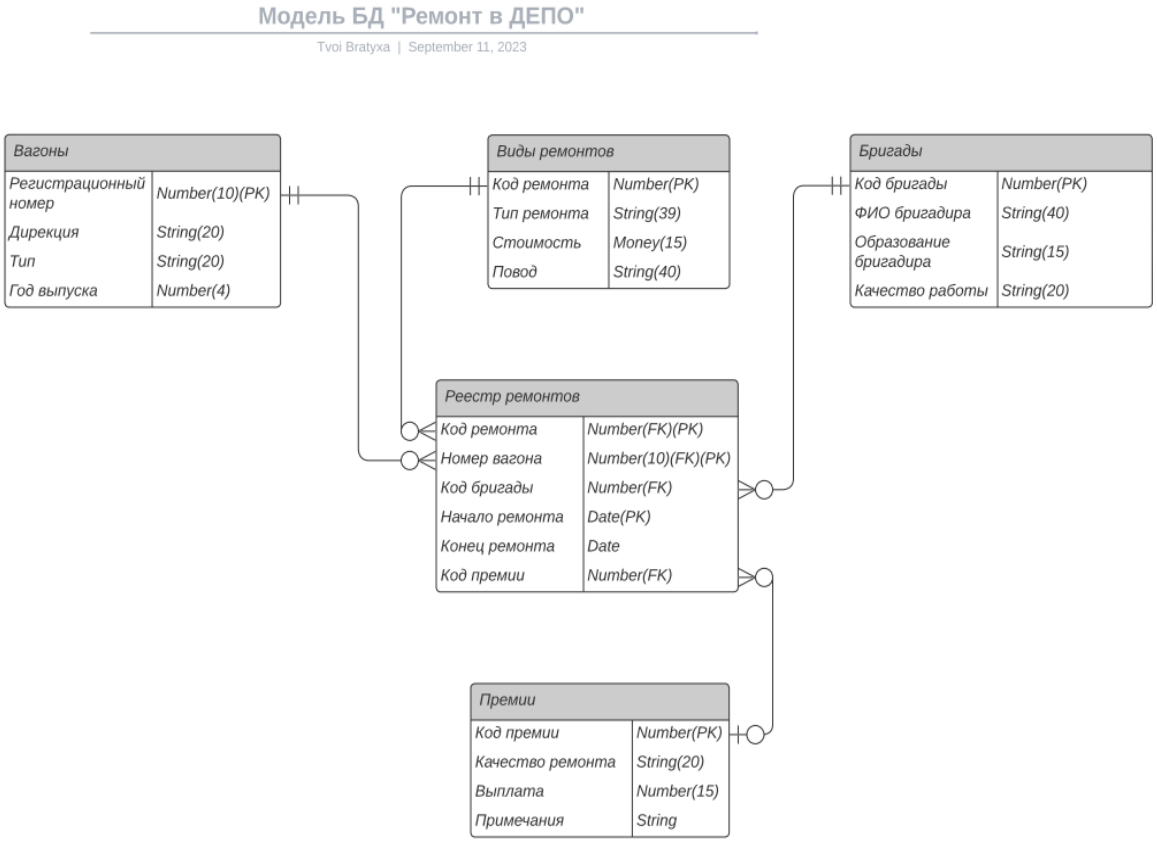


Рисунок 1 – ER-диаграмма для БД «Ремонт в ДЕПО»

Лабораторная работа № 2

«Создание новой базы данных в среде PostgreSQL»

Описание работы: При выполнении задания необходимо для заданной предметной области средствами PostgreSQL:

• создать новую базу данных;

• составить отчет.

Dashboard Properties SQL Statistics Dependencies Dependents Processes Untitled*

```sql
-- Database: DepotDB

-- DROP DATABASE IF EXISTS "DepotDB";

CREATE DATABASE "DepotDB"
    WITH
    OWNER = postgres
    ENCODING = 'UTF8'
    LC_COLLATE = 'Russian_Russia.1251'
    LC_CTYPE = 'Russian_Russia.1251'
    TABLESPACE = pg_default
    CONNECTION LIMIT = -1
    IS_TEMPLATE = False;

COMMENT ON DATABASE "DepotDB"
    IS 'прикладное программное обеспечение деятельности депо по ремонту пассажирских вагонов';
```

Рисунок 2 – SQL команда для создания БД

Лабораторная работа № 3

«Создание и связывание таблиц базы данных в среде PostgreSQL»

Описание работы: При выполнении задания необходимо для заданной предметной области:

- создать таблицы, определить поля таблиц и тип полей;
- определить связи между таблицами и ограничения целостности;
- составить отчет.

Query Query History

```sql
BEGIN;

CREATE TABLE IF NOT EXISTS public."Awards"
(
    "Award code" integer NOT NULL,
    "Quality job" character varying(20) COLLATE pg_catalog."default" NOT NULL,
    "Pay" numeric(15, 0) NOT NULL,
    "Notes" text COLLATE pg_catalog."default",
    CONSTRAINT "Awards_pkey" PRIMARY KEY ("Award code")
);

CREATE TABLE IF NOT EXISTS public."Brigades"
(
    "Brigade code" integer NOT NULL,
    "Foreman surname" character varying(40) COLLATE pg_catalog."default" NOT NULL,
    "Foreman forename" character varying(40) COLLATE pg_catalog."default" NOT NULL,
    "Foreman patronymic" character varying(40) COLLATE pg_catalog."default",
    "Foreman education" character varying(40) COLLATE pg_catalog."default" NOT NULL,
    "Quality job" character varying(20) COLLATE pg_catalog."default",
    CONSTRAINT "Brigades_pkey" PRIMARY KEY ("Brigade code")
);

CREATE TABLE IF NOT EXISTS public."Carriages"
(
    "Registration number" numeric(10, 0) NOT NULL,
    "Direction" character varying(20) COLLATE pg_catalog."default" NOT NULL,
    "Type" character varying(20) COLLATE pg_catalog."default" NOT NULL,
    "Issue year" numeric(4, 0) NOT NULL,
    CONSTRAINT "Carriages_pkey" PRIMARY KEY ("Registration number")
);
```

Total rows: 0 of 0

Рисунок 3 – SQL команды для создания таблиц (часть 1)

```
CREATE TABLE IF NOT EXISTS public."Repair register"
(
    "Repair code" integer NOT NULL,
    "Carriage number" numeric(10, 0) NOT NULL,
    "Brigade code" integer NOT NULL,
    "Repair start" date NOT NULL,
    "Repair end" date NOT NULL,
    "Award code" integer,
    CONSTRAINT "Repair register_pkey" PRIMARY KEY ("Repair code", "Carriage number", "Repair start")
);

CREATE TABLE IF NOT EXISTS public."Repair types"
(
    "Repair code" integer NOT NULL,
    "Repair type" character varying(39) COLLATE pg_catalog."default" NOT NULL,
    "Price" money NOT NULL,
    "Reason" character varying(40) COLLATE pg_catalog."default",
    CONSTRAINT "Repair types_pkey" PRIMARY KEY ("Repair code")
);

ALTER TABLE IF EXISTS public."Repair register"
    ADD CONSTRAINT "Repair register_Award code_fkey" FOREIGN KEY ("Award code")
    REFERENCES public."Awards" ("Award code") MATCH SIMPLE
    ON UPDATE NO ACTION
    ON DELETE NO ACTION
    NOT VALID;
```

Рисунок 4 – SQL команды для создания таблиц (часть 2)

```
ALTER TABLE IF EXISTS public."Repair register"
    ADD CONSTRAINT "Repair register_Brigade code_fkey" FOREIGN KEY ("Brigade code")
    REFERENCES public."Brigades" ("Brigade code") MATCH SIMPLE
    ON UPDATE NO ACTION
    ON DELETE NO ACTION
    NOT VALID;


ALTER TABLE IF EXISTS public."Repair register"
    ADD CONSTRAINT "Repair register_Carriage number_fkey" FOREIGN KEY ("Carriage number")
    REFERENCES public."Carriages" ("Registration number") MATCH SIMPLE
    ON UPDATE NO ACTION
    ON DELETE NO ACTION
    NOT VALID;


ALTER TABLE IF EXISTS public."Repair register"
    ADD CONSTRAINT "Repair register_Repair code_fkey" FOREIGN KEY ("Repair code")
    REFERENCES public."Repair types" ("Repair code") MATCH SIMPLE
    ON UPDATE NO ACTION
    ON DELETE NO ACTION;

END;
```

Рисунок 5 – SQL команды для создания таблиц (часть 3)

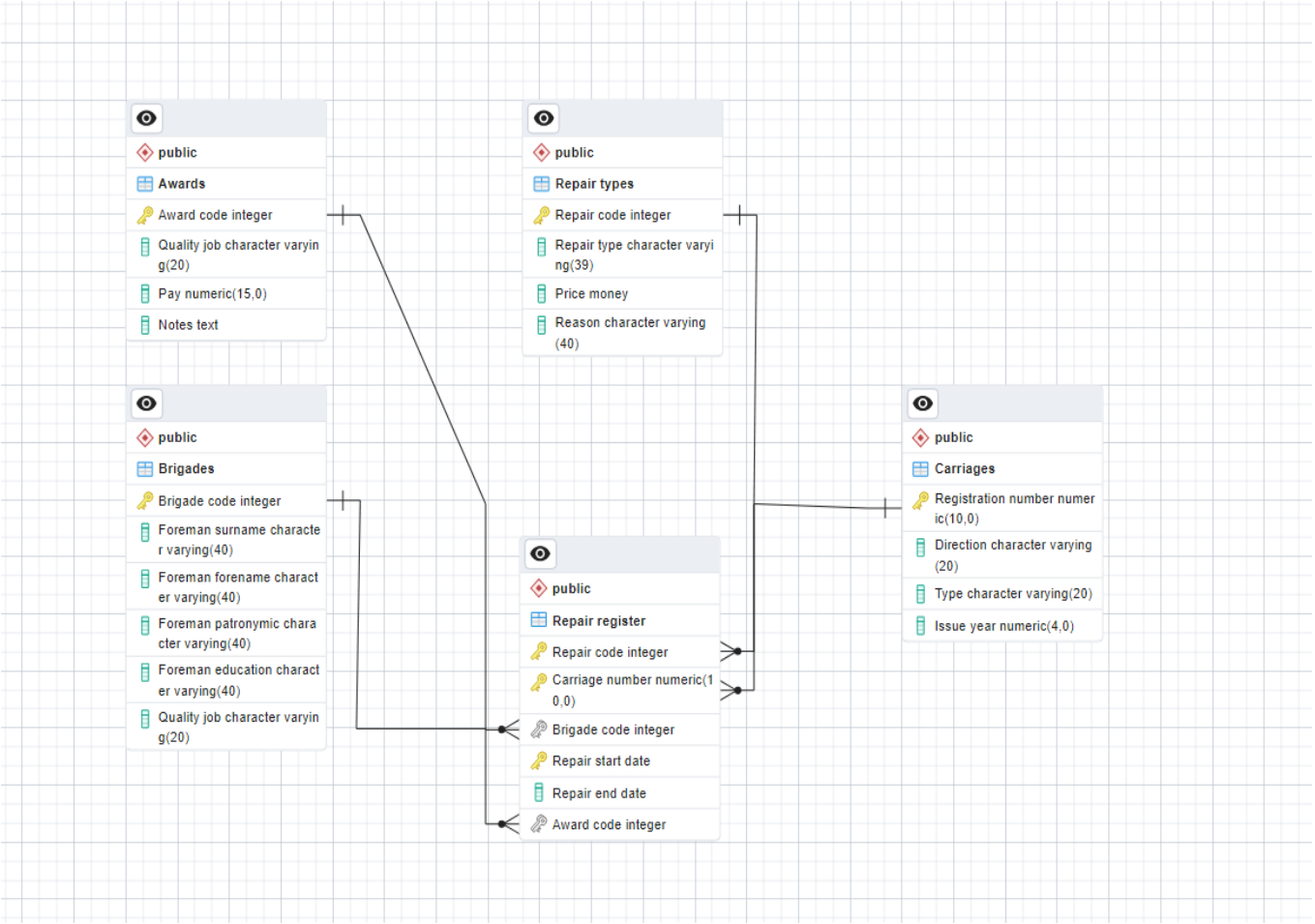


Рисунок 6 – Схема БД

Лабораторная работа № 4

«Вставка, удаление и обновление данных»

Описание работы: При выполнении задания, используя инструкции SQL:

- заполнить согласованными данными таблицы базы данных;
- при необходимости исправить введенную информацию;
- составить отчет.

Query | Query History

```
INSERT INTO "Brigades" ("Brigade code", "Foreman surname", "Foreman forename", "Foreman patronymic", "Foreman education", "Quality job")
VALUES (1, 'Евдокимов', 'Александр', 'Никитич', 'Осмотрщик-ремонтник вагонов', 'отличное'),
       (2, 'Марков', 'Алексей', 'Олегович', 'Специалист по обслуживанию вагонов', 'отличное'),
       (3, 'Егоров', 'Марк', 'Борисович', 'Ремонтник вагонов', 'низкое'),
       (4, 'Никольский', 'Кирилл', 'Павлович', 'Осмотрщик-ремонтник вагонов', 'среднее'),
       (5, 'Алексеев', 'Лука', 'Богданович', 'Оператор по ремонту вагонов', 'среднее');
```

Рисунок 7 – Вставка данных в таблицу «Бригады»

Query Query History

```
SELECT * FROM public."Brigades"
ORDER BY "Brigade code" ASC
```

Data Output Messages Notifications

	Brigade code [PK] integer	Foreman surname character varying (40)	Foreman forename character varying (40)	Foreman patronymic character varying (40)	Foreman education character varying (40)	Quality job character varying (20)
1	1	Евдокимов	Александр	Никитич	Осмотрщик-ремонтник вагонов	отличное
2	2	Марков	Алексей	Олегович	Специалист по обслуживанию вагонов	отличное
3	3	Егоров	Марк	Борисович	Ремонтник вагонов	низкое
4	4	Никольский	Кирилл	Павлович	Осмотрщик-ремонтник вагонов	среднее
5	5	Алексеев	Лука	Богданович	Оператор по ремонту вагонов	среднее

Рисунок 8 – Вывод данных из таблицы «Бригады»

Query Query History

```
INSERT INTO "Awards" ("Award code", "Quality job", "Pay", "Notes")
VALUES (1, 'отличное', 5000, 'отлично сделанная работа без замечаний'),
       (2, 'хорошое', 3000, 'хорошая работа'),
       (3, 'среднее', 1500, 'нормально сделанная работа, но с оставшимися неисправностями'),
       (4, 'низкое', 0, 'плохо сделанная работа, все неисправности остались'),
       (5, 'ужасное', 0, 'работа только ухудшила состояние вагона');
```

Рисунок 9 – Вставка данных в таблицу «Премии»

Query Query History

```
select * from "Awards"
```

Data Output Messages Notifications

	Award code [PK] integer	Quality job character varying (20)	Pay numeric (15)	Notes text
1	1	отличное	5000	отлично сделанная работа без замечаний
2	2	хорошое	3000	хорошая работа
3	3	среднее	1500	нормально сделанная работа, но с оставшимися неисправностя...
4	4	низкое	0	плохо сделанная работа, все неисправности остались
5	5	ужасное	0	работа только ухудшила состояние вагона

Рисунок 10 – Вывод данных из таблицы «Премии»

Query Query History

```
INSERT INTO "Carriages" ("Registration number", "Direction", "Type", "Issue year")
VALUES (1, 'ЦДМВ', 'крытый', 2002),
       (2, 'РЖД', 'цистерна', 1999),
       (3, 'РЖД', 'полувагон', 1977),
       (4, 'ЦДМВ', 'цистерна', 2011),
       (5, 'РЖД', 'цистерна', 2021);
```

Рисунок 11 – Вставка данных в таблицу «Вагоны»

Query | Query History

```
1  select * from "Carriages"
```

Data Output

	Registration number [PK] numeric (10)	Direction character varying (20)	Type character varying (20)	Issue year numeric (4)
1	1	ЦДМВ	крытый	2002
2	2	РЖД	цистерна	1999
3	3	РЖД	полувагон	1977
4	4	ЦДМВ	цистерна	2011
5	5	РЖД	цистерна	2021

Рисунок 12 – Вывод данных из таблицы «Вагоны»

Query | Query History

```
1  INSERT INTO "Repair types" ("Repair code", "Repair type", "Price", "Reason")
2  VALUES (1, 'Деповский', 25000, 'Пробег 450 тыс.км или 2 г. эксплуатации'),
3         (2, 'ТО-2', 10000, 'Подготовка к зимним или летним условиям'),
4         (3, 'ТО-3', 15000, 'Пробег 150 тыс.км или 1 г. эксплуатации'),
5         (4, 'Текущий', 2000, 'Круглосуточно');
```

Рисунок 13 – Вставка данных в таблицу «Виды ремонтов»

Query | Query History

```
1  select * from "Repair types"
```

Data Output

	Repair code [PK] integer	Repair type character varying (39)	Price money	Reason character varying (40)
1	1	Деповский	25 000,00 ?	Пробег 450 тыс.км или 2 г. эксплуатации
2	2	ТО-2	10 000,00 ?	Подготовка к зимним или летним условиям
3	3	ТО-3	15 000,00 ?	Пробег 150 тыс.км или 1 г. эксплуатации
4	4	Текущий	2 000,00 ?	Круглосуточно

Рисунок 14 – Вывод данных из таблицы «Виды ремонтов»

Query Query History

```
INSERT INTO "Repair register" ("Repair code", "Carriage number", "Brigade code", "Repair start", "Repair end", "Award code")
VALUES (4, 3, 2, '2023-06-19', '2023-06-20', 3),
       (4, 5, 1, '2023-08-20', '2023-08-24', 4),
       (4, 1, 5, '2023-09-29', '2023-09-30', 4),
       (1, 2, 4, '2021-11-10', '2021-11-19', 1),
       (1, 2, 4, '2019-09-1', '2019-09-3', 1),
       (4, 4, 3, '2023-08-21', '2023-08-22', 2),
       (2, 5, 1, '2022-12-11', '2022-12-22', 4),
       (3, 5, 1, '2021-11-29', '2021-12-1', 3),
       (1, 3, 3, '2022-10-30', '2022-11-2', 2),
       (4, 1, 3, '2023-09-11', '2023-09-12', 2),
       (2, 1, 2, '2023-03-12', '2023-03-19', 2),
       (4, 2, 4, '2023-08-12', '2023-08-13', 1),
       (4, 3, 1, '2023-09-5', '2023-09-6', 2),
       (4, 3, 1, '2023-09-7', '2023-09-8', 3);
```

Рисунок 15 – Вставка данных в таблицу «Реестр ремонтов»

Query Query History

```
select * from "Repair register"
```

Data Output

	Repair code [PK] integer	Carriage number [PK] numeric (10)	Brigade code integer	Repair start [PK] date	Repair end date	Award code integer
1	4	3	2	2023-06-19	2023-06-20	3
2	4	5	1	2023-08-20	2023-08-24	4
3	4	1	5	2023-09-29	2023-09-30	4
4	1	2	4	2021-11-10	2021-11-19	1
5	1	2	4	2019-09-01	2019-09-03	1
6	4	4	3	2023-08-21	2023-08-22	2
7	2	5	1	2022-12-11	2022-12-22	4
8	3	5	1	2021-11-29	2021-12-01	3
9	1	3	3	2022-10-30	2022-11-02	2
10	4	1	3	2023-09-11	2023-09-12	2
11	2	1	2	2023-03-12	2023-03-19	2
12	4	2	4	2023-08-12	2023-08-13	1
13	4	3	1	2023-09-05	2023-09-06	2
14	4	3	1	2023-09-07	2023-09-08	3

Рисунок 16 – Вывод данных из таблицы «Реестр ремонтов»

Лабораторная работа № 5

«Работа с массивами и типами JSON»

Описание работы: Необходимо для созданной базы данных:

• добавить таблицу со столбцом, содержащим массив значений;

• добавить таблицу со столбцом, содержащим значения в формате JSON;

• составить отчет.

```sql
CREATE TABLE IF NOT EXISTS public."Members brigade experience"
(
    "Brigade code" integer NOT NULL,
    "Members experience" numeric[] NOT NULL,
    "Count members" integer NOT NULL,
    CONSTRAINT "Members brigade experience_pkey" PRIMARY KEY ("Brigade code"),
    CONSTRAINT "Brigade_code_fkey" FOREIGN KEY ("Brigade code")
        REFERENCES public."Brigades" ("Brigade code") MATCH SIMPLE
        ON UPDATE NO ACTION
        ON DELETE NO ACTION
)

TABLESPACE pg_default;

ALTER TABLE IF EXISTS public."Members brigade experience"
    OWNER to postgres;
```

Рисунок 17 – Создание таблицы «Опыт работы участников бригад»

Query Query History

```sql
INSERT INTO "Members brigade experience" ("Brigade code", "Members experience", "Count members")
VALUES (1, '{6,4,6,8,1,2}'::numeric[], 6),
       (2, '{1,4,5,11,1,3,4,10}'::numeric[], 8),
       (3, '{4,1,1,2}'::numeric[], 4),
       (4, '{6,4,8,7,9}'::numeric[], 5),
       (5, '{1,15,11,8,1,8}'::numeric[], 7);
```

Рисунок 18 – Вставка данных в таблицу «Опыт работы участников бригад»

Query Query History

```sql
SELECT * FROM public."Members brigade experience"
ORDER BY "Brigade code" ASC
```

Data Output

	Brigade code [PK] integer	Members experience numeric[]	Count members integer
1	1	{6,4,6,8,1,2}	6
2	2	{1,4,5,11,1,3,4,10}	8
3	3	{4,1,1,2}	4
4	4	{6,4,8,7,9}	5
5	5	{1,15,11,8,1,8}	7

Рисунок 19 – Вывод данных из таблицы «Опыт работы участников бригад»

```sql
CREATE TABLE IF NOT EXISTS public."Railways"
(
    "Railway code" integer NOT NULL,
    "Carriage code" integer NOT NULL,
    "Requisites" jsonb NOT NULL,
    CONSTRAINT "Railways_pkey" PRIMARY KEY ("Railway code", "Carriage code"),
    CONSTRAINT "Carriage_code_fkey" FOREIGN KEY ("Carriage code")
        REFERENCES public."Carriages" ("Registration number") MATCH SIMPLE
        ON UPDATE NO ACTION
        ON DELETE NO ACTION
)

TABLESPACE pg_default;

ALTER TABLE IF EXISTS public."Railways"
    OWNER to postgres;
```

Рисунок 20 – Создание таблицы «Железные дороги»

Query | Query History

```sql
INSERT INTO "Railways" ("Railway code", "Carriage code", "Requisites")
VALUES (1, 1, '{"Банк": "Сбербанк",
        "ИНН": "7708503727",
        "Юридический адрес": "г. Москва, ул. Новая Басманная, 2/1"}'
        ::jsonb),
       (2, 4, '{"Банк": "Сбербанк",
        "ИНН": "100088574",
        "Юридический адрес": "г. Москва, ул. Ленина, 17"}'
        ::jsonb),
       (3, 3, '{"Банк": "Росбанк",
        "ИНН": "020540003431",
        "Юридический адрес": "г. Москва, ул. Демьяна, 22/2"}'
        ::jsonb),
       (4, 2, '{"Банк": "ВТБ",
        "ИНН": "1104000230",
        "Юридический адрес": "г. Москва, ул. Водников, 45"}'
        ::jsonb),
       (5, 5, '{"Банк": "Сбербанк",
        "ИНН": "7705501211",
        "Юридический адрес": "г. Москва, ул. Декабристов, 3"}'
        ::jsonb);
```

Рисунок 21 – Вставка данных в таблицу «Железные дороги»

Query | Query History

```sql
SELECT * FROM public."Railways"
ORDER BY "Railway code" ASC, "Carriage code" ASC
```

Data Output

	Railway code [PK] integer	Carriage code [PK] integer	Requisites jsonb
1	1	1	{"ИНН": "7708503727", "Банк": "Сбербанк", "Юридический адрес": "г. Москва, ул. Новая Басманная, 2/1"}
2	2	4	{"ИНН": "100088574", "Банк": "Сбербанк", "Юридический адрес": "г. Москва, ул. Ленина, 17"}
3	3	3	{"ИНН": "020540003431", "Банк": "Росбанк", "Юридический адрес": "г. Москва, ул. Демьяна, 22/2"}
4	4	2	{"ИНН": "1104000230", "Банк": "ВТБ", "Юридический адрес": "г. Москва, ул. Водников, 45"}
5	5	5	{"ИНН": "7705501211", "Банк": "Сбербанк", "Юридический адрес": "г. Москва, ул. Декабристов, 3"}

Рисунок 22 – Вывод данных из таблицы «Железные дороги»

Лабораторная работа № 6

«Создание запросов на выборку»

Описание работы: Необходимо для созданной базы данных:

• построить запрос на выборку с использованием какого-либо соединения;

• построить запрос на выборку, содержащий вложенный запрос;

• составить отчет.

Query | Query History

```
1  select "Brigade code", sum("Pay") as totalPay
2  from "Repair register" join "Awards"
3  on "Repair register"."Award code" = "Awards"."Award code"
4  group by "Brigade code"
5  order by totalPay asc;
```

Data Output

	Brigade code integer	totalpay numeric
1	5	0
2	2	4500
3	1	6000
4	3	9000
5	4	15000

Рисунок 23 – Создание запроса для вывода общего числа заработанных премий бригад

Query | Query History

```
1  select * from "Carriages"
2  where 2 < (select count("Carriage number") from "Repair register"
3            where "Carriages"."Registration number" = "Repair register"."Carriage number"
4            group by "Carriage number")
```

Data Output

	Registration number [PK] numeric (10)	Direction character varying (20)	Type character varying (20)	Issue year numeric (4)
1	1	ЦДМВ	крытый	2002
2	2	РЖД	цистерна	1999
3	3	РЖД	полувагон	1977
4	5	РЖД	цистерна	2021

Рисунок 24 – Создание запроса для вывода вагонов, которые больше 2 раз были в ремонте

Лабораторная работа № 7

«Представления»

Описание работы: Необходимо для созданной базы данных:

• создать несколько простых и материализованных представлений и включить их в базу данных;

• сформировать запросы к этим представлениям;

• составить отчет.

Query | Query History

```
1  create view totalPay_of_brigades as
2  select "Brigade code", sum("Pay") as totalPay
3  from "Repair register" join "Awards"
4  on "Repair register"."Award code" = "Awards"."Award code"
5  group by "Brigade code"
6  order by totalPay asc;
```

Рисунок 25 – Создание простого представления для вывода общих сумм премий бригад

Query | Query History

```
1  select * from totalPay_of_brigades
2  where totalPay > 0;
```

Data Output

	Brigade code integer	totalpay numeric
1	2	4500
2	1	6000
3	3	9000
4	4	15000

Рисунок 26 – Создание запроса для вывода данных из представления

Query | Query History

```
1  create view avgPay_of_brigades as
2  select "Brigade code", avg("Pay") as avgPay, min("Pay") as minPay, max("Pay") as maxPay
3  from "Repair register" join "Awards"
4  on "Repair register"."Award code" = "Awards"."Award code"
5  group by "Brigade code"
6  order by avgPay desc, maxPay desc, minPay desc;
```

Рисунок 27 – Создание простого представления для вывода средней, минимальной и максимальной премии бригад

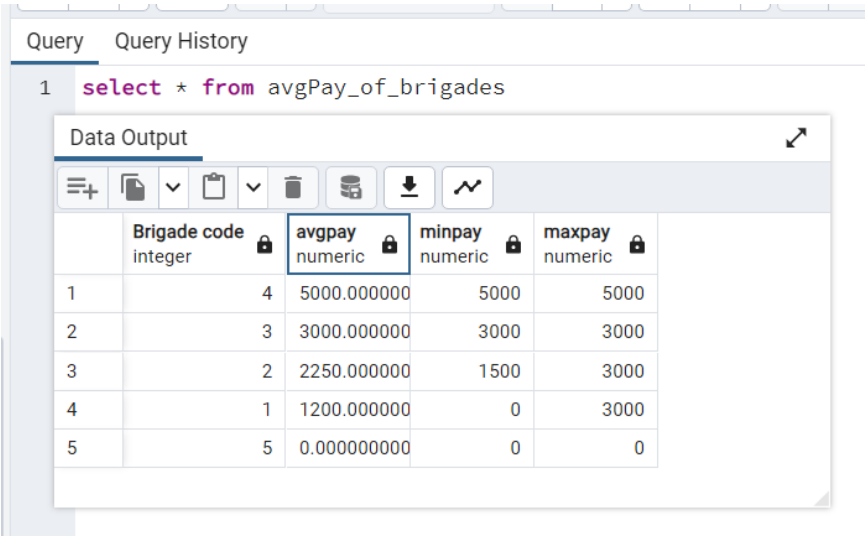


Рисунок 28 – Создание запроса для вывода данных из представления

```
Query  Query History
1  create materialized view carriages_count_repair as
2  select "Carriage number", count("Carriage number") as count_repair from "Repair register"
3  group by "Carriage number"
4  order by count_repair asc;
```

Рисунок 29 – Создание материализованного представления для вывода общего числа ремонтов вагонов

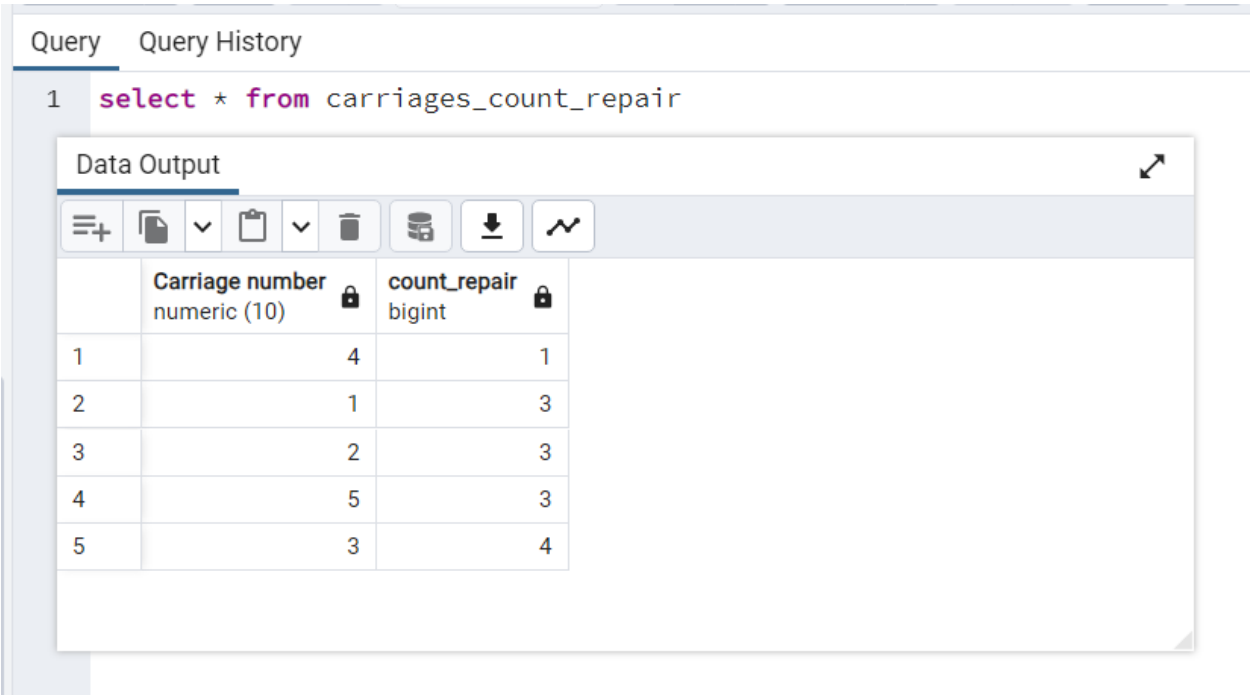


Рисунок 30 – Создание запроса для вывода данных из представления

Query | Query History

```
1  create materialized view often_repair_type as
2  select "Repair register"."Repair code", "Repair types"."Repair type",
3  count("Repair register"."Repair code") as count_type from "Repair register"
4  join "Repair types"
5  on "Repair register"."Repair code" = "Repair types"."Repair code"
6  group by "Repair register"."Repair code", "Repair types"."Repair type"
```

Рисунок 31 – Создание материализованного представления для вывода частоты видов ремонтов

Query | Query History

```
1  select * from often_repair_type
2
```

Data Output

	Repair code integer	Repair type character varying (39)	count_type bigint
1	4	Текущий	8
2	2	ТО-2	2
3	3	ТО-3	1
4	1	Деповский	3

Рисунок 32 – Создание запроса для вывода данных из представления

Лабораторная работа № 8

«Функции и процедуры»

Описание работы: Необходимо для созданной базы данных:

• написать несколько хранимых процедур и включить их в базу данных;

• составить отчет.

Query | Query History

```
1  create or replace procedure brigade_insert
2          (Brigade_code integer, Foreman_surname character varying,
3           Foreman_forename character varying, Foreman_patronymic character varying,
4           Foreman_education character varying, Quality_job character varying)
5  language sql as $$
6      insert into "Brigades"("Brigade code", "Foreman surname", "Foreman forename",
7           "Foreman patronymic", "Foreman education", "Quality job")
8      values (Brigade_code, Foreman_surname,
9           Foreman_forename, Foreman_patronymic,
10          Foreman_education, Quality_job);
11 $$;
```

Рисунок 33 – Создание процедуры для добавления данных в таблицу «Бригады»

Query | Query History

```
1  call brigade_insert(6, 'Иванов', 'Иван', 'Иванович', 'Ремонтник вагонов', 'хорошее');
```

Рисунок 34 – Вызов процедуры

Query | Query History

```
1  SELECT * FROM public."Brigades"
2  ORDER BY "Brigade code" ASC
```

Data Output

	Brigade code [PK] integer	Foreman surname character varying (40)	Foreman forename character varying (40)	Foreman patronymic character varying (40)	Foreman education character varying (40)	Quality job character varying (20)
1	1	Евдокимов	Александр	Никитич	Осмотрщик-ремонтник вагонов	отличное
2	2	Марков	Алексей	Олегович	Специалист по обслуживанию вагонов	отличное
3	3	Егоров	Марк	Борисович	Ремонтник вагонов	низкое
4	4	Никольский	Кирилл	Павлович	Осмотрщик-ремонтник вагонов	среднее
5	5	Алексеев	Лука	Богданович	Оператор по ремонту вагонов	среднее
6	6	Иванов	Иван	Иванович	Ремонтник вагонов	хорошее

Рисунок 35 – Вывод данных из таблицы «Бригады»

Query | Query History

```
1  create or replace procedure brigade_update
2          (Brigade_code integer, Foreman_surname character varying,
3           Foreman_forename character varying, Foreman_patronymic character varying,
4           Foreman_education character varying, Quality_job character varying)
5  language sql as $$
6      update "Brigades" set "Brigade code" = Brigade_code, "Foreman surname" = Foreman_surname,
7              "Foreman forename" = Foreman_forename, "Foreman patronymic" = Foreman_patronymic,
8              "Foreman education" = Foreman_education, "Quality job" = Quality_job
9      where "Brigade code" = Brigade_code;
10 $$;
```

Рисунок 36 – Создание процедуры для изменения данных в таблице «Бригады»

Query Query History

```
1  call brigade_update(6, 'Сидоров', 'Сергей', 'Александрович', 'Ремонтник вагонов', 'отличное');
```

Рисунок 37 – Вызов процедуры

Query Query History

```
1  SELECT * FROM public."Brigades"
2  ORDER BY "Brigade code" ASC
```

Data Output

	Brigade code [PK] integer	Foreman surname character varying (40)	Foreman forename character varying (40)	Foreman patronymic character varying (40)	Foreman education character varying (40)	Quality job character varying (20)
1	1	Евдокимов	Александр	Никитич	Осмотрщик-ремонтник вагонов	отличное
2	2	Марков	Алексей	Олегович	Специалист по обслуживанию вагонов	отличное
3	3	Егоров	Марк	Борисович	Ремонтник вагонов	низкое
4	4	Никольский	Кирилл	Павлович	Осмотрщик-ремонтник вагонов	среднее
5	5	Алексеев	Лука	Богданович	Оператор по ремонту вагонов	среднее
6	6	Сидоров	Сергей	Александрович	Ремонтник вагонов	отличное

Рисунок 38 – Вывод данных из таблицы «Бригады»

Query Query History

```
1  create or replace procedure brigade_delete(Brigade_code integer)
2  language sql as $$
3      delete from "Brigades"
4      where "Brigade code" = Brigade_code;
5  $$;
```

Рисунок 39 – Создание процедуры для удаления данных в таблице «Бригады»

Query Query History

```
1  call brigade_delete(6);
```

Рисунок 40 – Вызов процедуры

Query Query History

```
1 SELECT * FROM public."Brigades"
2 ORDER BY "Brigade code" ASC
```

Data Output

	Brigade code [PK] integer	Foreman surname character varying (40)	Foreman forename character varying (40)	Foreman patronymic character varying (40)	Foreman education character varying (40)	Quality job character varying (20)
1	1	Евдокимов	Александр	Никитич	Осмотрщик-ремонтник вагонов	отличное
2	2	Марков	Алексей	Олегович	Специалист по обслуживанию вагонов	отличное
3	3	Егоров	Марк	Борисович	Ремонтник вагонов	низкое
4	4	Никольский	Кирилл	Павлович	Осмотрщик-ремонтник вагонов	среднее
5	5	Алексеев	Лука	Богданович	Оператор по ремонту вагонов	среднее

Рисунок 41 – Вывод данных из таблицы «Бригады»

Query Query History

```
1  create type brigade_type as (code integer, name character varying, surname character varying,
2                               patronymic character varying, education character varying,
3                               quality_job character varying);
4
5  create or replace function brigade_select() returns setof brigade_type
6  language sql as $$
7      select row("Brigade code", "Foreman surname", "Foreman forename",
8           "Foreman patronymic", "Foreman education", "Quality job")::brigade_type
9      from "Brigades";
10 $$;
```

Рисунок 42 – Создание функции для вывода данных из таблицы «Бригады»

Query Query History

```
1 select * from brigade_select();
```

Data Output

	code integer	name character varying	surname character varying	patronymic character varying	education character varying	quality_job character varying
1	1	Евдокимов	Александр	Никитич	Осмотрщик-ремонтник вагонов	отличное
2	2	Марков	Алексей	Олегович	Специалист по обслуживанию вагонов	отличное
3	3	Егоров	Марк	Борисович	Ремонтник вагонов	низкое
4	4	Никольский	Кирилл	Павлович	Осмотрщик-ремонтник вагонов	среднее
5	5	Алексеев	Лука	Богданович	Оператор по ремонту вагонов	среднее

Рисунок 43 – Вывод данных из функции

Лабораторная работа № 9

«Транзакции»

Описание работы: Необходимо для созданной базы данных:

• создать транзакцию с точкой сохранения в режиме явных транзакций, произвести откат к точке сохранения и фиксацию;

• составить отчет.

Query | Query History

```
1  begin;
2  insert into "Repair types"("Repair code", "Repair type", "Price", "Reason")
3  values (5, 'Ежегодный', 8000, 'Ежегодный ремонт');
4  savepoint point_one;
5  insert into "Repair types"("Repair code", "Repair type", "Price", "Reason")
6  values (6, 'Экстренный', 100000, 'В случае критической поломки');
7  savepoint point_one;
8  insert into "Repair types"("Repair code", "Repair type", "Price", "Reason")
9  values (7, 'Профилактика', 1000, 'Проверка на наличие неисправностей');
10 rollback to savepoint point_one;
11 commit;
12 select * from "Repair types";
```

Data Output

	Repair code [PK] integer	Repair type character varying (39)	Price money	Reason character varying (40)
1	1	Деповский	25 000,00 ?	Пробег 450 тыс.км или 2 г. эксплуатации
2	2	ТО-2	10 000,00 ?	Подготовка к зимним или летним усло…
3	3	ТО-3	15 000,00 ?	Пробег 150 тыс.км или 1 г. эксплуатации
4	4	Текущий	2 000,00 ?	Круглосуточно
5	5	Ежегодный	8 000,00 ?	Ежегодный ремонт
6	6	Экстренный	100 000,00 ?	В случае критической поломки

Рисунок 44 – Создание транзакции с точками сохранения, откатом, фиксацией и вывод данных из таблицы

Лабораторная работа № 10

«Курсоры»

Описание работы: Необходимо для созданной базы данных:

• решить несколько задач с использованием курсоров, включив курсор в хранимую процедуру или функцию;

• составить отчет.

Query Query History

```
1  create or replace procedure railways_select()
2  language 'plpgsql' as $$
3  declare
4  rail_id integer; carr_id integer; requisites jsonb;
5  cur cursor for select * from "Railways";
6  begin
7  open cur;
8  loop
9  fetch cur into rail_id, carr_id, requisites;
10 exit when not found;
11 raise notice 'Дорога: %; Прикрепленный вагон: %; Реквизиты дороги: %', rail_id, carr_id, requisites;
12 end loop;
13 close cur;
14 end;
15 $$;
```

Рисунок 45 – Создание процедуры с курсором для вывода данных из таблицы «Железные дороги»

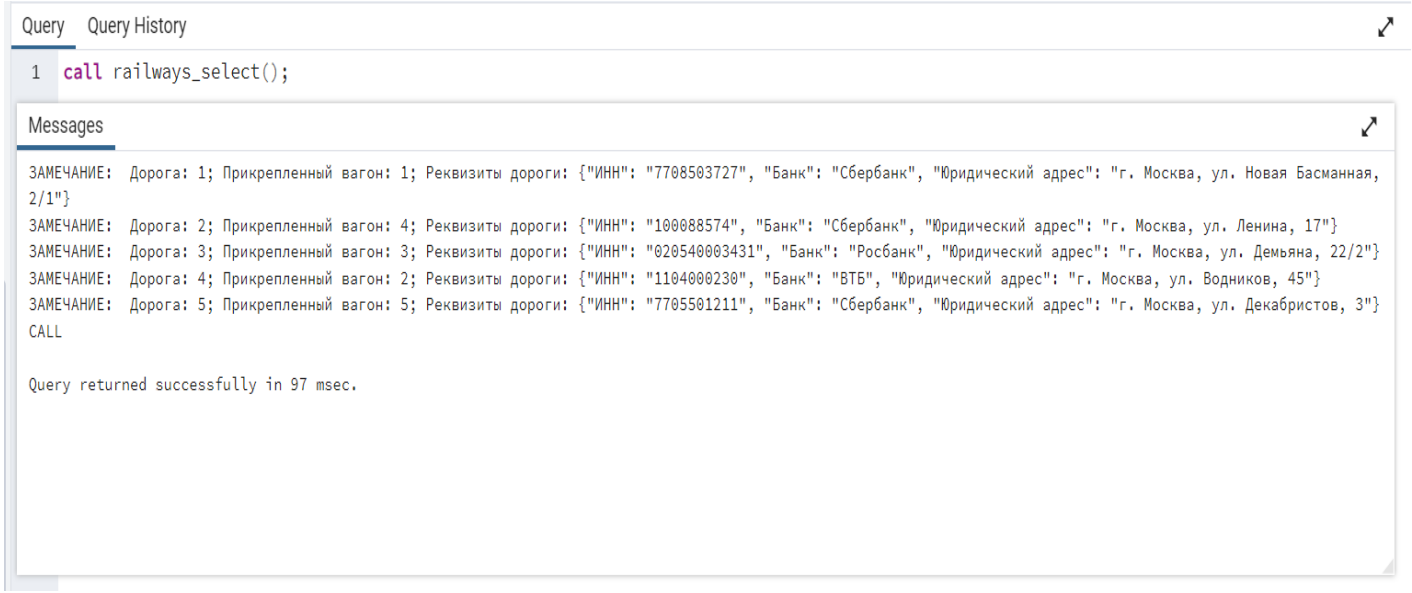


Рисунок 46 – Вызов процедуры

```
CREATE TABLE IF NOT EXISTS public."Quality repair september"
(
    "Brigade code" integer NOT NULL,
    "Foreman surname" character varying COLLATE pg_catalog."default" NOT NULL,
    "Carriage number" integer NOT NULL,
    "Carriage type" character varying COLLATE pg_catalog."default" NOT NULL,
    "Repair code" integer NOT NULL,
    "Repair type" character varying COLLATE pg_catalog."default" NOT NULL,
    "Quality" character varying COLLATE pg_catalog."default" NOT NULL,
    "Award" integer,
    CONSTRAINT table_pkey PRIMARY KEY ("Brigade code", "Carriage number", "Repair code")
);
```

Рисунок 47 – Создание таблицы «Качественные ремонты сентября»

Query Query History

```
create or replace procedure quality_repair_sep()
language 'plpgsql' as $$
declare
brig_id integer; surname character varying; carr_code integer;
carr_type character varying; repair_code integer; repair_type character varying;
quality character varying; award integer;
cur cursor for select "Repair register"."Brigade code", "Brigades"."Foreman surname", "Repair register"."Carriage number",
"Carriages"."Type", "Repair register"."Repair code", "Repair types"."Repair type", "Awards"."Quality job", "Awards"."Pay"
from "Repair register"
join "Brigades" on "Brigades"."Brigade code" = "Repair register"."Brigade code"
join "Awards" on "Awards"."Award code" = "Repair register"."Award code"
join "Repair types" on "Repair types"."Repair code" = "Repair register"."Repair code"
join "Carriages" on "Carriages"."Registration number" = "Repair register"."Carriage number"
where "Repair register"."Repair start"::text LIKE '%-09-%' and
("Repair register"."Award code" = 1 or "Repair register"."Award code" = 2);
begin
open cur;
loop
fetch cur into brig_id, surname, carr_code, carr_type, repair_code, repair_type, quality, award;
exit when not found;
insert into "Quality repair september"("Brigade code", "Foreman surname", "Carriage number",
                                       "Carriage type", "Repair code", "Repair type", "Quality", "Award")
values (brig_id, surname, carr_code, carr_type, repair_code, repair_type, quality, award);
end loop;
close cur;
end;
$$;
```

Рисунок 48 – Создание процедуры с курсором для вывода качественно сделанных ремонтов за сентябрь в созданную таблицу

Query Query History

```
call quality_repair_sep();
select * from "Quality repair september";
```

Data Output

	Brigade code [PK] integer	Foreman surname character varying	Carriage number [PK] integer	Carriage type character varying	Repair code [PK] integer	Repair type character varying	Quality character varying	Award integer
1	4	Никольский	2	цистерна	1	Деповский	отличное	5000
2	3	Егоров	1	крытый	4	Текущий	хорошое	3000
3	1	Евдокимов	3	полувагон	4	Текущий	хорошое	3000

Рисунок 49 – Вызов процедуры и вывод данных из таблицы

Лабораторная работа № 11

«Триггеры»

Описание работы: Необходимо для созданной базы данных:

• написать два триггера, срабатывающих при модификации данных;

• составить отчет.

Query Query History

```
1  create or replace function change_railways() returns trigger
2  language 'plpgsql' as $$
3  begin
4  if (tg_op = 'DELETE') then
5  raise notice '% % %', user, tg_op, old."Railway code";
6  else raise notice '% % % % %', user, tg_op, new."Railway code", new."Carriage code", new."Requisites";
7  end if;
8  return null;
9  end;
10 $$;
```

Рисунок 50 – Создание процедуры для вывода информации об операции

Query Query History

```
1  create trigger ch_railway
2  after insert or update or delete on "Railways"
3  for each row execute function change_railways();
```

Рисунок 51 – Создание триггера для изменения таблицы «Железные дороги»

Query Query History

```
1  insert into "Railways"
2  values (6, 1, '{
3    "ИНН": 7708503727,
4    "Банк": "Сбербанк",
5    "Юридический адрес": "г. Москва, ул. Новая Басманная, 2/1"
6  }');
```

Messages

```
ЗАМЕЧАНИЕ:  postgres INSERT 6 1 {"ИНН": 7708503727, "Банк": "Сбербанк", "Юридический адрес": "г. Москва, ул.
Новая Басманная, 2/1"}
INSERT 0 1

Query returned successfully in 144 msec.
```

Рисунок 52 – Вызов триггерной процедуры

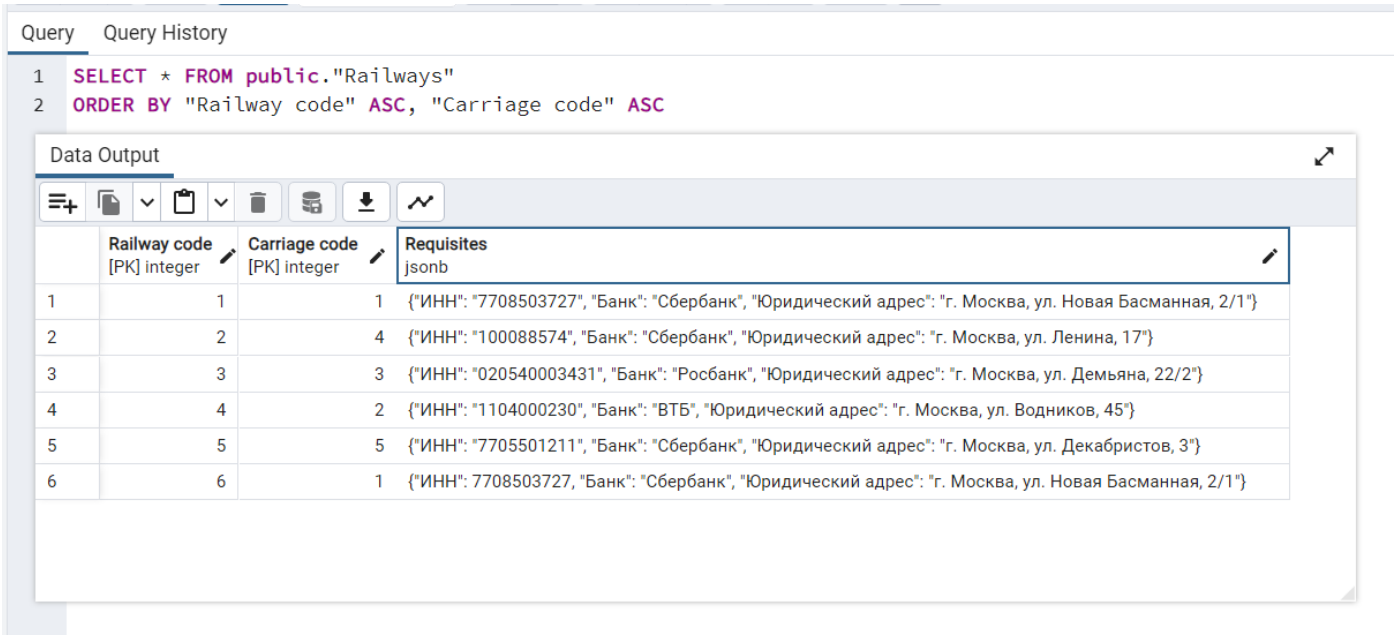


Рисунок 53 – Вывод данных из таблицы

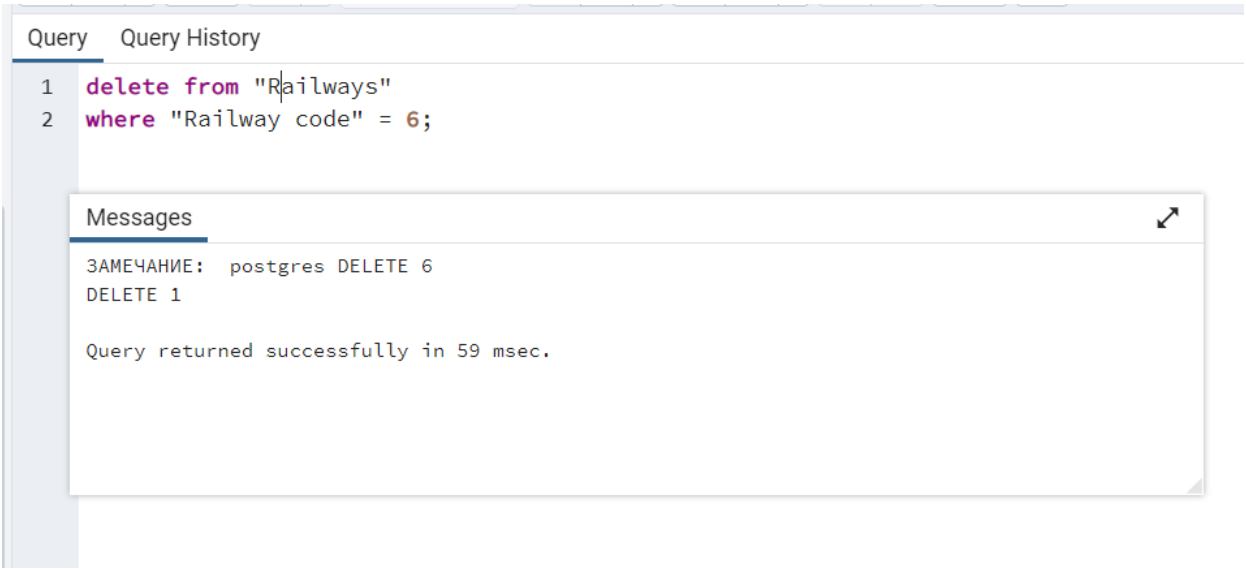


Рисунок 54 – Вызов триггерной процедуры при удалении данных

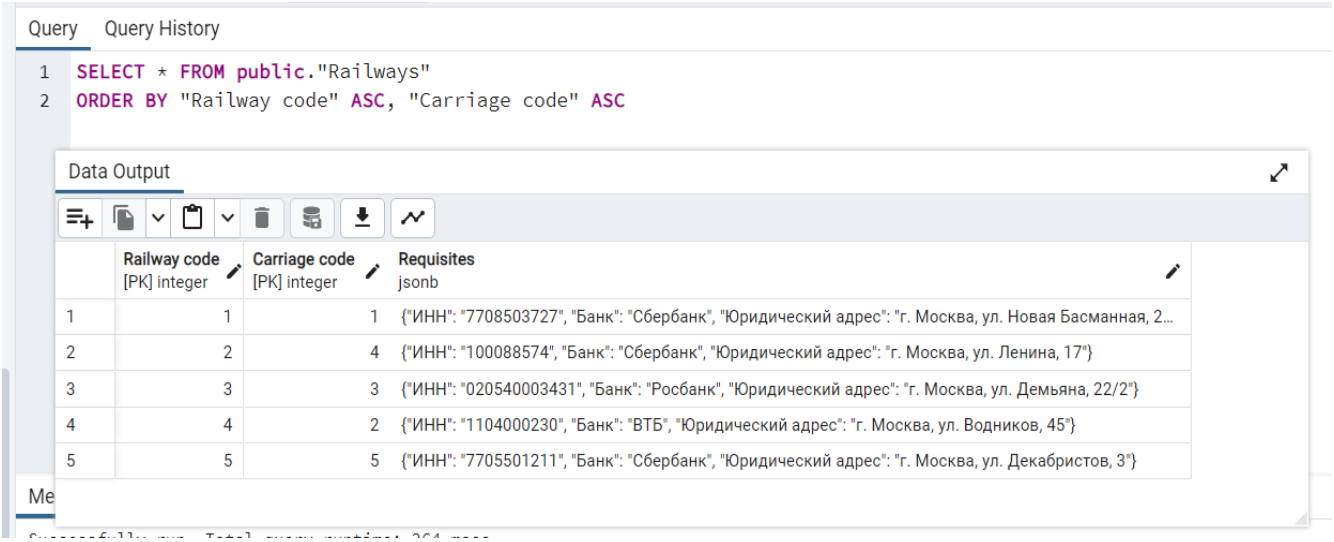


Рисунок 55 – Вывод данных из таблицы

Query | Query History

```
1  create or replace function dont_change_awards() returns trigger
2  language 'plpgsql' as $$
3  begin
4  raise exception 'Запрещено добавлять или удалять премии';
5  return new;
6  end;
7  $$;
8
9  create trigger ban_awards
10 before insert or delete on "Awards"
11 for each row execute function dont_change_awards();
```

Рисунок 56 – Создание процедуры и триггера для запрета добавления/удаления записей в таблице «Премии»

Query | Query History

```
1  insert into "Awards"
2  values (6, 'нормальное', 2000, 'лучше среднего качества, но далеко от совершенства');
3  SELECT * FROM public."Awards"
4  ORDER BY "Award code" ASC;
```

Messages

```
ERROR:  Запрещено добавлять или удалять премии
CONTEXT:  функция PL/pgSQL dont_change_awards(), строка 3, оператор RAISE

ОШИБКА:  Запрещено добавлять или удалять премии
SQL state: P0001
```

Рисунок 57 – Вызов триггерной процедуры

Query | Query History

```
1  SELECT * FROM public."Awards"
2  ORDER BY "Award code" ASC
```

Data Output

	Award code [PK] integer	Quality job character varying (20)	Pay numeric (15)	Notes text
1	1	отличное	5000	отлично сделанная работа без замечаний
2	2	хорошое	3000	хорошая работа
3	3	среднее	1500	нормально сделанная работа, но с оставшимися неисправностя...
4	4	низкое	0	плохо сделанная работа, все неисправности остались
5	5	ужасное	0	работа только ухудшила состояние вагона

Рисунок 58 – Вывод данных из таблицы

Заключение

В результате выполнения лабораторных работ были рассмотрены все основные функции СУБД “PostgreSQL”, изучен синтаксис языка PL/pgSQL и SQL, создана реляционная БД по учету ремонта вагонов при помощи приложения pgAdmin, которое имеет мощный графический интерфейс, позволяющий решать задачи администрирования, отображать объекты баз данных и выполнять запросы на языке SQL.

Список использованных источников

1. Домбровская, Г. Оптимизация запросов в PostgreSQL: полное руководство по созданию эффективных запросов / Г. Домбровская, Б. Новиков, А. Бейликова. – Москва: ДМК Пресс, 2022. – 277 с.

2. Кригель, А. SQL. Библия пользователя / А. Кригель, Б. Трухнов. – 2-е изд. – Москва: Вильямс, 2010. – 744 с.

3. Лузанов, П. В. Postgres. Первое знакомство / П. В. Лузанов, Е. В. Рогов, И. В. Лёвшин. – 6-е изд. – Москва: Постгрес Профессиональный, 2020. – 178 с.

4. Моргунов, Е. П. PostgreSQL. Основы языка SQL: учеб. пособие / Е. П. Моргунов. – Санкт-Петербург: БХВ-Петербург, 2018. – 335 с.

5. Новиков, Б. А. Основы технологий баз данных: учеб. пособие / Б. А. Новиков, Е. А. Горшкова, Н. Г. Графеева. – 2-е изд. – Москва: ДМК Пресс, 2020. – 582 с.

6. Рогов, Е. В. PostgreSQL 15 изнутри / Е. В. Рогов. – Москва: ДМК Пресс, 2023. – 662 с.

7. PostgreSQL Server Programming / U. Dar, H. Krosing, J. Mlodgenski, K. Roybal. – 2nd Edition Extend. – Birmingham: Packt Publishing, 2015. – 508 p.

8. PostgreSQL. Documentation. 9th February 2023: PostgreSQL 15.2, 14.7, 13.10, 12.14, and 11.19 Released. – URL: https://www.postgresql.org/docs/ (дата обращения: 30.03.2023).

9. PostgresPro. Документация к PostgreSQL 15.2. – URL: https://postgrespro.ru/docs/postgresql/15/ (дата обращения: 13.09.2023).

10. pgAdmin 4. Documentation. Release 7.1. The pgAdmin Development Team May 03, 2023. – URL: https://www.pgadmin.org/docs/pgadmin4/latest/index.html (дата обращения: 13.09.2023).